Lasers & Mirrors

Paul Brandstetter
Matrikelnummer: k12310371

Abgabedatum: 2. April 2026

Programmieren 2

1 Quellcode

Der Code ist in 4 Files aufgeteilt.

Kleine Hilfsfunktionen sind in Aux.h zu finden:

```
#include <cmath>
#include "Drawing.h"

#ifndef _Aux_

#define _Aux_

using namespace std;

/*
 * equals(a,b,eps)
 * checks whether a and b are equal up to tolerance eps
 */
bool equals(double a, double b, double eps) {
   if (abs(a - b) < eps) {
      return true;
   }
   return false;
}

/*
 * toPolar(x,y,r,a)
 * writes polar coordinates of (x,y) into (r,a)
 */
void toPolar(double x, double y, double &r, double &a) {
   a = atan2(y, x);
   r = sqrt(x * x + y * y);
}

/*
 * toCartesian(r,a,x,y)
 * writes cartesian represantation or (r,a) into x,y)
 */
void toCartesian(double r, double a, double &x, double &y) {
   x = r * cos(a);
   y = r * sin(a);
}

/*
 * color(PosX,PosY)
 * returns rainbow colors based on PosX and PosY
 */
```

```
int color(int PosX, int PosY) {
   int r = 256 * PosX / compsys::getWidth();
   int g = 256 * PosY / compsys::getHeight();
   int b = min(256 * 256 - r * r - g * g, 0);
   r = r * 0x010000;
   g = g * 0x000100;
   b = floor(sqrt(b));
   return r + g + b;
}

#endif
```

Die Klasse Segment inklusive ihrer Methoden befinden sich in Segment.h:

```
#include <cmath>
#include <stdexcept>
#include <iostream>
#include "Aux.h"

#ifndef _Segment_

#define _Segment_

class Segment {

public:
   double x0;
   double y0;
   double x1;
   double y1;

   /*
    * Self explanatory constructors...
    */
   Segment() {
   }
   Segment(double StartX, double StartY, double EndX, double EndY) {
      x0 = StartX;
      y0 = StartY;
      x1 = EndX;
      y1 = EndY;
   }

   /*
    * operator<<
    * output is of form
    * x0 y0 x1 y1
```

```
     * with:
     * (x0,y0) coordinates of starting point
     * (x1,y1) coordinates of endpoint
     * all coordinates are rounded down to nearest integer
     */
    friend std::ostream& operator<<(std::ostream &os, const Segment &s) {
       os << floor(s.x0) << " " << floor(s.y0) << " " << floor(s.x1) << " "
             << floor(s.y1);
       return os;
    }

    /*
     * length()
     * returns length of segment measured in euclidean distance
     */
    const double length();

    /*
     * run()
     * returns horizontal difference of Start and Endpoint
     */
    const double run();

    /*
    * rise()
    * returns vertical difference of Start and Endpoint
    */
    const double rise();

    /*
     * angle();
     * returns cw angle of line segment
     */
    const double angle();

    /*
     * collides(other)
     * returns true if this and other collide
     * in that case, the endpoint of this is set to the point of collision
     */
    bool collides(Segment &other);
};

const double Segment::length() {
    return std::sqrt(std::pow((x1 - x0), 2) + std::pow((y1 - y0), 2));
}
```

```
const double Segment::run() {
   return x1 - x0;
}

const double Segment::rise() {
     return y1 - y0;
  }

const double Segment::angle()  {
   return std::atan2(rise(), run());
}

bool Segment::collides(Segment &other) {
   if (angle() == other.angle()) {
      //if this happens, we don't want to break the program with an exception.
      //but we cannot calculate a unique collision point either.
      //Therefore we just pretend, that they don't colllide.
      //The odds are 0% anyway
      return false;
   }
   //Solve a + mu*b = c + lambda*d with
   //a = (x0,y0)
   //c = (other.x0,other.y0)
   //b = (run(),rise())
   //d = (other.run),other.rise())
   double det = run() * other.rise() - rise() * other.run(); //determinant of matrix
   double mu = (other.rise() * (other.x0 - x0)
         - other.run() * (other.y0 - y0)) / det;
   double lambda = (run() * (other.y0 - y0) - rise() * (other.x0 - x0))
         / det;
   if ((0 <= mu) && (mu <= 1) && (-1 <= lambda) && (lambda <= 0)) {
      x1 = x0 + mu * run();
      y1 = y0 + mu * rise();
      return true;
   }
   return false;
}

#endif
```

Die Funktionen, die im Skelettcode benötigt werden befinden sich in Functions.h:

```
#include "Segment.h"
#include "Drawing.h"
#include "Aux.h"
#include <cmath>
```

```
#include <iostream>
#include <random>
#include <fstream>

using namespace compsys;
using namespace std;

/*
 * reflectRayOld(n,mirrors,ray,ray0)
 * n is number of mirrors.
 * checks which mirror ray is going to collide with first
 * calculates the collision point
 * sets the end of ray to the collision point
 * doesn't yet set ray0 (which is the reflected ray) correctly
 */
void reflectRayOld(int n, Segment *&mirrors, Segment &ray, Segment &ray0);

/*
 * reflectRay(n,mirrors,ray,ray0)
 * n is number of mirrors.
 * checks which mirror ray is going to collide with first
 * calculates the collision point
 * sets the end of ray to the collision point
 * starts ray0 at collision point and sets the trajectory correctly.
 * ray0 will have a length of 1 (up to some numerical imprecision)
 */
void reflectRay(int n, Segment *&mirrors, Segment &ray, Segment &ray0);

/*
 * drawRay(ray)
 * continually draws a ray from start to endoint of ray
 * draws a dot at endpoint.
 * Color is based on position, cycles through most colors.
 */

void drawRay(Segment &ray);

/*
 * init(argc, argv, ray, n, mirrors)
 * initializes ray and mirrors
 * if called with no arguments, (argc = 1)
 * it generates a random n from 5 to 15
 * and creates 1 to 11 random mirrors as well as 1 for each wall of canvas.
 * also creates a random ray.
 *
 * if called with an input file as an argument (argc = 2)
```

```
 * it creates mirrors and ray accordingly.
 *
 * Format:
 * x0 y0 x1 y1
 * n
 * x01 y01 x11 y11
 * ...
 * x0n y0n y1n y1n
 *
 * no matter what is called, will output the state at the beginnging to the console
 */
void init(int argc, const char **argv, Segment &ray, int &n,
      Segment *&mirrors);

/*
 * drawMirrors(n, mirrors)
 * draws n mirrors on screen:
 */
void drawMirrors(int n, Segment *&mirrors);

void reflectRayOld(int n, Segment *&mirrors, Segment &ray, Segment &ray0) {
   double VelocityX = (ray.run()) / ray.length();
   double VelocityY = (ray.rise()) / ray.length();
   ray.x1 = ray.x0 + VelocityX;
   ray.y1 = ray.y0 + VelocityY;
   while (true) {
      for (int i = 0; i < n; i++) {
         if (ray.collides(mirrors[i])) {
            ray0.x0 = 1;
            ray0.x1 = 100;
            ray0.y0 = 1;
            ray0.y1 = 100;
            return;
         }
      }
      ray.x1 += VelocityX;
      ray.y1 += VelocityY;
   }
}

void reflectRay(int n, Segment *&mirrors, Segment &ray, Segment &ray0) {
   double VelocityX = (ray.run()) / ray.length();
   double VelocityY = (ray.rise()) / ray.length();
   ray.x1 = ray.x0 + VelocityX;
   ray.y1 = ray.y0 + VelocityY;
   while (true) {
```

```
      for (int i = 0; i < n; i++) {
         if (ray.collides(mirrors[i])) {

            double r, a;
            toPolar(-VelocityX, -VelocityY, r, a);
            a -= mirrors[i].angle();
            toCartesian(r, a, VelocityX, VelocityY);
            VelocityX = -VelocityX;
            toPolar(VelocityX, VelocityY, r, a);
            a += mirrors[i].angle();
            toCartesian(r, a, VelocityX, VelocityY);

            //Ray 0 is instantly offset by a little
            //such that the ray doesn't hit the same mirror twice
            ray0.x0 = ray.x1 + VelocityX;
            ray0.y0 = ray.y1 + VelocityY;
            ray0.x1 = ray0.x0 + VelocityX;
            ray0.y1 = ray0.y0 + VelocityY;

            return;
         }
      }
      ray.x1 += VelocityX;
      ray.y1 += VelocityY;
   }
}

void drawRay(Segment &ray) {
   double S = 2;
   double VelocityX = S * (ray.run()) / ray.length();
   double VelocityY = S * (ray.rise()) / ray.length();
   double CurrentX = ray.x0;
   double CurrentY = ray.y0;
   for (int i = 0; i < ray.length() / S - 1; i++) {
      drawLine(CurrentX, CurrentY, CurrentX + VelocityX, CurrentY + VelocityY,
            color(CurrentX, CurrentY));
      CurrentX += VelocityX;
      CurrentY += VelocityY;
   }
   drawLine(CurrentX, CurrentY, ray.x1, ray.y1, color(CurrentX, CurrentY));
   fillEllipse(ray.x1 - 5, ray.y1 - 5, 10, 10, color(ray.x1, ray.y1), 0x000000);
}

void init(int argc, const char **argv, Segment &ray, int &n,
      Segment *&mirrors) {
   if (argc == 1) {
```

```
   //everything gets random numbers

   int W = getWidth();
   int H = getHeight();
   default_random_engine rng;
   random_device rand_dev;
   rng.seed(rand_dev());

   uniform_int_distribution<int> Number(5, 15);
   uniform_int_distribution<int> Xcoord(0, W);
   uniform_int_distribution<int> Ycoord(0, H);

   n = Number(rng);
   mirrors = new Segment[n];

   for (int i = 0; i < n - 4; i++) {
      mirrors[i] = Segment(Xcoord(rng), Ycoord(rng), Xcoord(rng),
            Ycoord(rng));
   }
   mirrors[n - 4] = Segment(0, 0, 0, H);
   mirrors[n - 3] = Segment(0, 0, W, 0);
   mirrors[n - 2] = Segment(0, H, W, H);
   mirrors[n - 1] = Segment(W, 0, W, H);

   ray = Segment(Xcoord(rng), Ycoord(rng), Xcoord(rng), Ycoord(rng));
} else {
   //code for reading file
   fstream in(argv[1], std::fstream::in);
   if (!in) {
      return;
   }
   int x0;
   int x1;
   int y0;
   int y1;
   in >> std::skipws;
   in >> x0 >> y0 >> x1 >> y1;
   ray = Segment(x0, y0, x1, y1);
   in >> n;
   mirrors = new Segment[n];
   for (int i = 0; i < n; i++) {
      in >> x0 >> y0 >> x1 >> y1;
      mirrors[i] = Segment(x0, y0, x1, y1);
   }
}
```

```
   cout << ray << std::endl;
   cout << n << std::endl;
   for (int i = 0; i < n; i++) {
      cout << mirrors[i] << std::endl;
   }
}

void drawMirrors(int n, Segment *&mirrors) {
   for (int i = 0; i < n; i++) {
      drawLine(mirrors[i].x0, mirrors[i].y0, mirrors[i].x1, mirrors[i].y1,
            0x008FFF);
   }
}
```

und zuletzt der Skelettcode in Main.cpp:

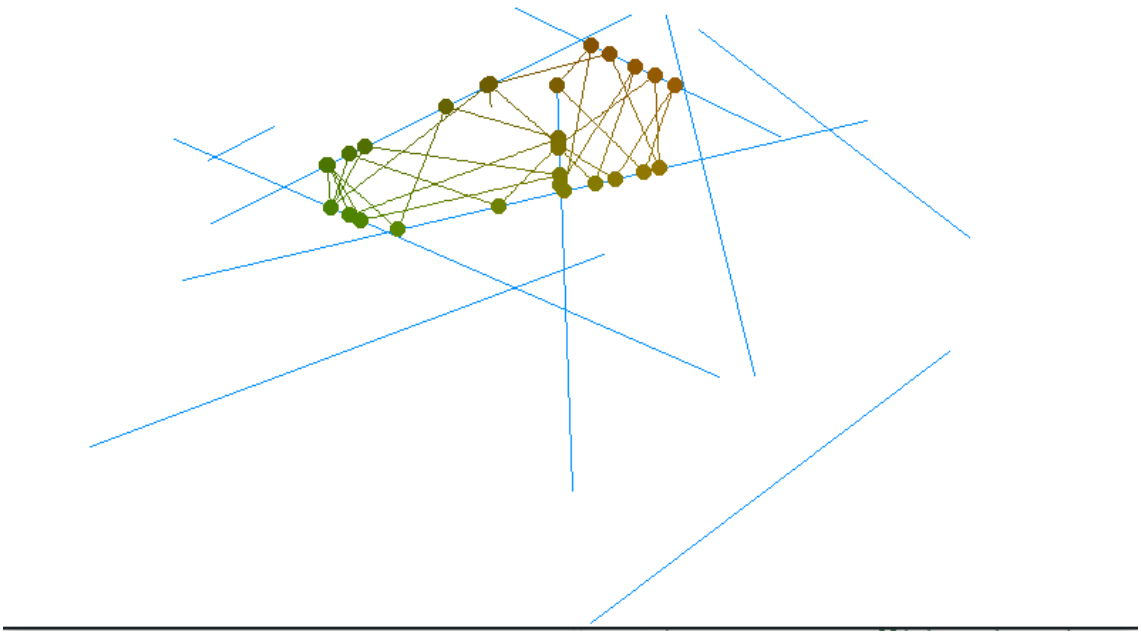
```
#include <Functions.h>
#include <iostream>
#include <cstdlib>
#include <cmath>
#include "Drawing.h"
#include "Segment.h"
#include <thread>
using namespace std;
using namespace compsys;

const int W = 799;
const int H = 599;
const int T = 30;

int main(int argc, const char *argv[]) {
   beginDrawing(W, H, "Laser", 0xFFFFFF);
   Segment ray;
   int n;
   Segment *mirrors;
   init(argc, argv, ray, n, mirrors);
   drawMirrors(n, mirrors);
   for (int i = 0; i < T; i++) {
      Segment ray0;
      reflectRay(n, mirrors, ray, ray0);
      drawRay(ray);
      ray = ray0;
   }
   delete[] mirrors;

   cout << "Close window to exit..." << endl;
   endDrawing();
```

```
    return 0;
}
```

2 Tests

Der Code wurde mit folgenden inputs getestet:

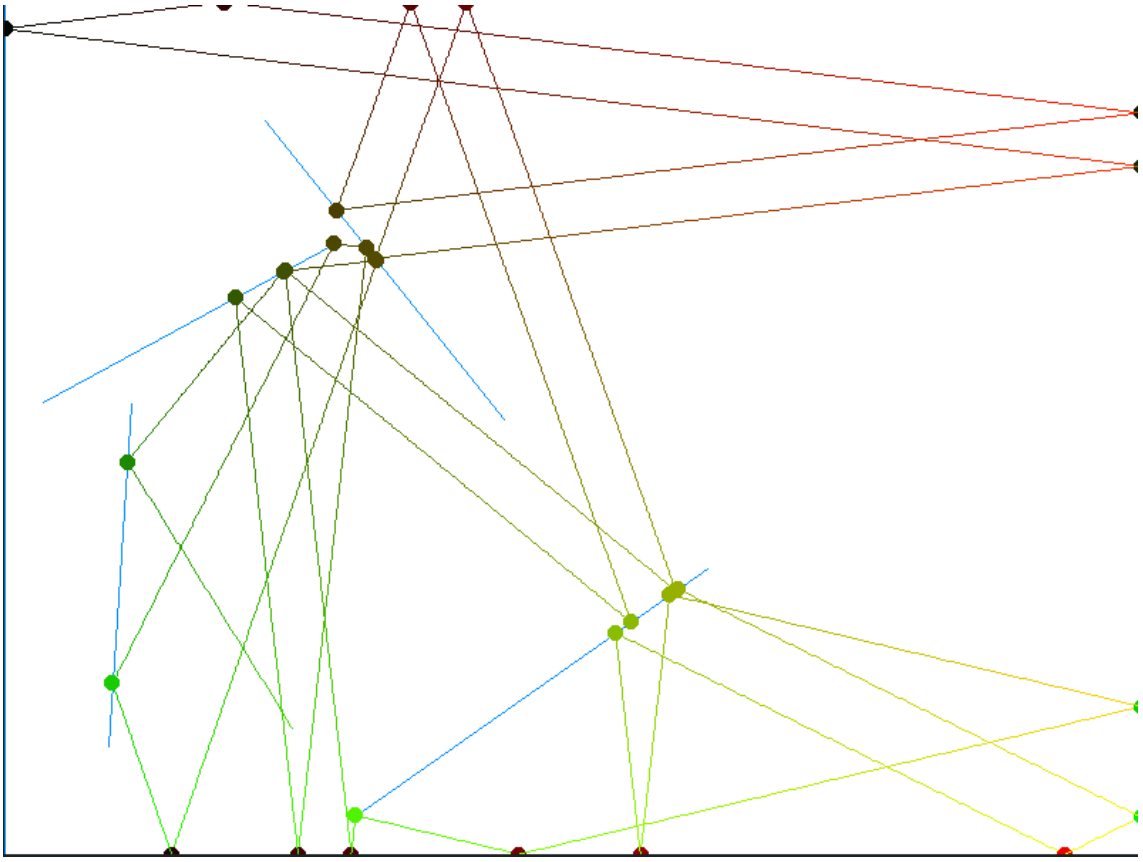
2.1 Kein Input



2.2 input.txt

Der Input war:

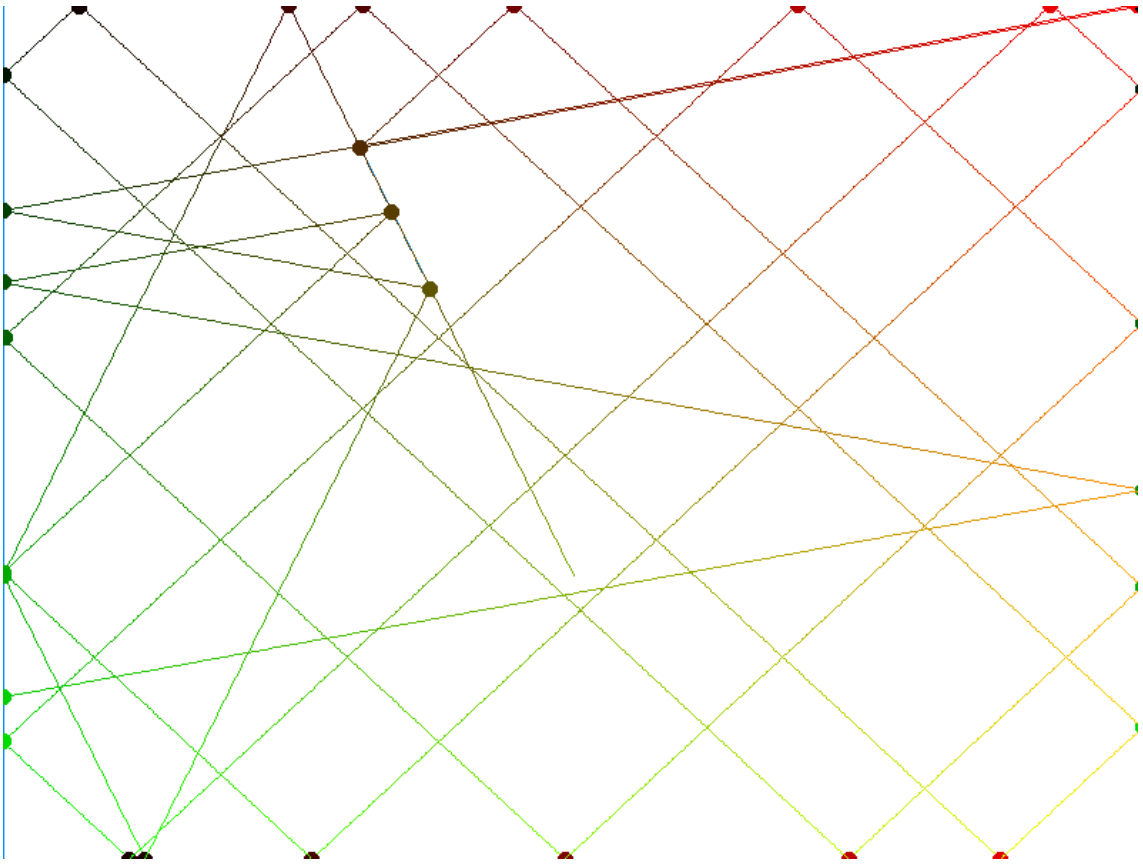
```
202 510 38 245
8
0 0 799 0
799 0 799 599
799 599 0 599
0 599 0 0
183 83 351 293
89 282 73 523
245 573 494 398
233 169 27 281
```



2.3 input1.txt

Der Input war:

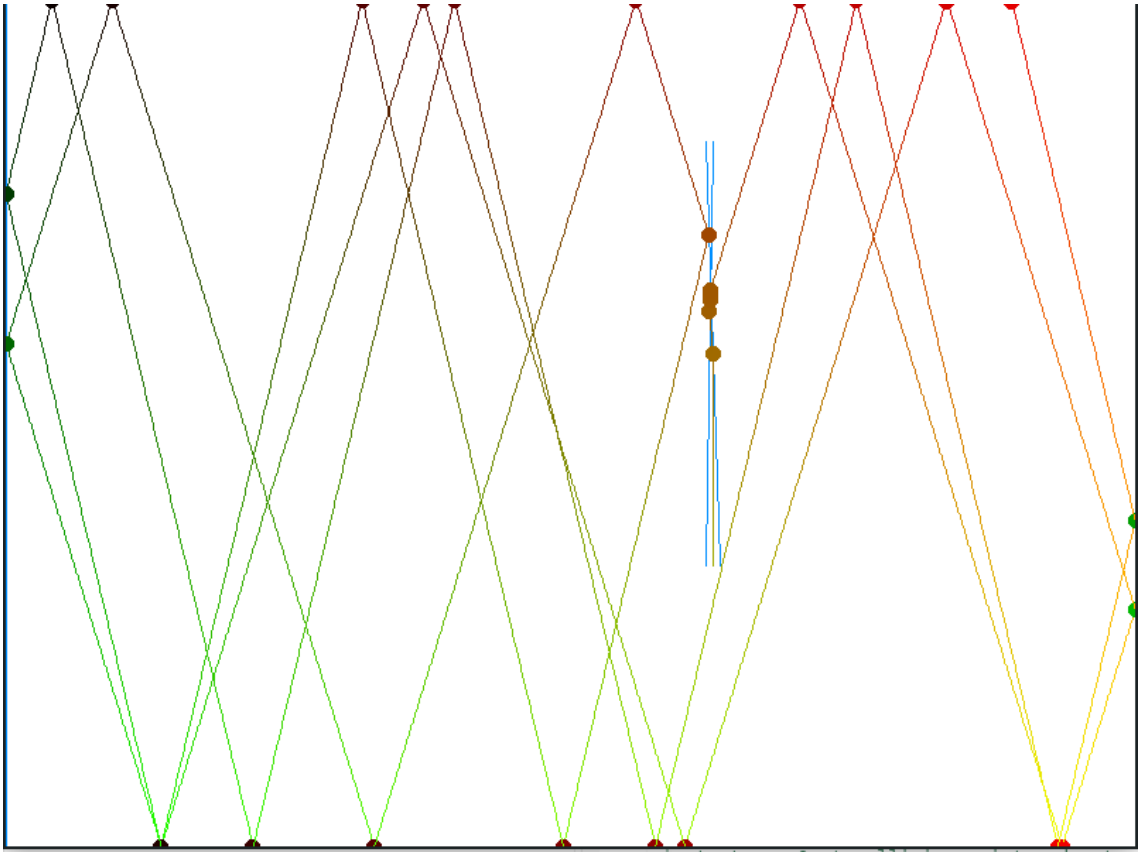
```
400 400 300 200
5
0 0 799 0
799 0 799 599
799 599 0 599
0 599 0 0
300 200 250 100
```



2.4 input2.txt

Der Input war:

```
500 400 500 100
6
0 0 799 0
799 0 799 599
799 599 0 599
0 599 0 0
505 400 495 100
495 400 500 100
```



Probleme Der Laser bewegt sich nicht immer mit derselben Geschwindigkeit. mit jedem Aufruf von drawLine wird ein gleichlanges Stück gezeichnet. Aber ich denke, der Befehl drawLine braucht einfach immer unterschiedlich lang. Warum das ist, kann ich nicht sagen, ich kann mir kaum vorstellen, dass so enorme Berechnungen durchgeführt werden.

Wenn der Laser und ein Spiegel denselben Winkel haben, lässt sich kein eindeutiger Aufprallpunkt feststellen. in diesem Fall habe ich einfach gesagt, dass keine Collision passiert. Das ist im Fall input1.txt ausprobiert worden. Der Laser geht einfach durch. Stattdessen hätte ich eine

Fehlermeldung ausgeben können, aber ich habe mich dagegen entschieden, da es besser ist, wenn das Programm einfach weiterläuft.

Wenn zwei Spiegel nah aneinander sind, kommt es zu eigenartigem Verhalten. Im Fall input2.exe sieht man beispielsweise, dass der Strahl aus den beiden Spiegeln ausbricht, obwohl er das theoretisch nicht sollte. Auch, wenn der Laser in eine Ecke zwischen zwei Spiegeln trifft, wird es ähnliches Verhalten geben.

Als ich reflectRay programmiert habe, hatte ich das issue, dass der Strahl zweimal hintereinander denselben Spiegel trifft und deshalb in irgendeine Richtung reflektiert wird. Das habe ich gefixt, indem ich den Beginn des zweiten, reflektierten Strahls etwas verschoben habe. Ich denke daher kommt das beschriebene Verhalten. Eleganter wäre es eventuell gewesen, hätte ich irgendwie dafür gesorgt, dass die Collision detection den jeweils letzten Spiegel ignoriert. Vermutlich wäre das mit einer globalen Variable oder so gegangen, aber ich hab das nicht gemacht.